

1.1) Exploring basic Linux commands and Shell [programs.ln](#)

Solution: Basic Linux Terminal commands

Sl.no	Commands	Discription
1.	ls	Displays information about files in the current directory.
2.	pwd	Displays the current working directory.
3.	mkdir	Creates a directory.
4.	cd	To navigate between different folders.
5.	rmdir	Removes empty directories from the directory lists.
6.	cp	Copy files from one directory to another.
7.	mv	Rename and Replace the files
8.	rm	Delete files
9.	uname	Command to get basic information about the OS
10.	locate	Find a file in the database.
11.	touch	Create empty files
12.	ln	Create shortcuts to other files
13.	cat	Display file contents on the terminal
14.	clear	Clear terminal
15.	ps	Display the processes in the terminal
16.	man	Access the manual for all Linux commands
17.	grep	Search for a specific string in an output
18.	echo	Print string or text to the terminal
19.	wget	download files from the internet.
20.	whoami	Displays the current user's name
21.	sort	sort the file content

	cal	View the Calendar in the terminal
23.	whereis	View the exact location of any command typed after this command
24.	df	Check the details of the file system
25.	wc	Check the lines, word count, and characters in a file using different options

grep – Search for a specific string in output/file

Syntax:

```
grep "text" filename
```

Example Program:

```
echo -e "Apple\nBanana\nMango" > fruits.txt
grep "Banana" fruits.txt
```

Output:

```
Banana
```

echo – Print text to terminal

Syntax:

```
echo "text"
```

Example:

```
echo "Hello Ubuntu"
```

Output:

```
Hello Ubuntu
```

wget – Download files from internet

Syntax:

wget URL

Example:

wget https://example.com/sample.txt

Downloads `sample.txt` to your current directory.

whoami – Display current username

Syntax:

whoami

Example:

whoami

Output Example:

priyanka

sort – Sort file contents

Syntax:

sort filename

Example Program:

```
echo -e "Zebra\nApple\nMango" > names.txt
sort names.txt
```

Output:

Apple
Mango
Zebra

cal – View calendar in terminal

Syntax:

cal

Example:

cal 2 2026

Displays calendar of February 2026.

whereis – Find location of a command

Syntax:

whereis command_name

Example:

whereis python

Output Example:

python: /usr/bin/python3

df – Check file system details

Syntax:

```
df
```

Example:

```
df -h
```

`-h` shows size in human-readable format (GB/MB).

wc – Count lines, words, characters

Syntax:

```
wc filename
```

Example Program:

```
echo -e "Hello\nUbuntu\nLinux" > sample.txt  
wc sample.txt
```

Output Example:

```
3 3 20 sample.txt
```

(3 lines, 3 words, 20 characters)

man – Access the manual of a command

Syntax:

```
man command_name
```

Example:

```
man ls
```

Opens the manual page for `ls` command.
Press `q` to quit the manual.

```
# Create a file
echo -e "Ubuntu\nLinux\nOperating System" > os.txt
```

```
# View file
cat os.txt
```

```
# Count words
wc os.txt
```

```
# Sort file
sort os.txt
```

```
# Search text
grep "Linux" os.txt
```

```
# Show current user
whoami
```

```
# Show disk usage
df -h
```

```
# Show calendar
cal
```

```
# Locate grep command
whereis grep
```

1.2) Execute the C program using Command Line Arguments.

- i. Add numbers**
- ii. Find the minimum from the given numbers**
- iii. Find the average of the given number**

Solution:

i. Add_num.c /* Create this file */

```
#include <stdio.h>
#include <stdlib.h>
int Add_num(int nParams, char * argv[]);
int main(int argc, char *argv[])
{
```

```

printf("Calculate the sum of numbers entered by the user!!!\n");
if(argc < 2)
{
printf("Error: Insufficient parameters!!!\n");
printf("Give at least one or more valid integer numbers as parameters.\n");
return -1;
}

int nParams = argc -1;
int sum = 0;
for(int i = 0; i < argc; i++)
{
printf(" argv[%d] = %s\n", i, argv[i]);
}
sum = Add_num(nParams, argv);
return 0;
}

int Add_num (int nParams, char * argv[])
{
char *pParam;
int sum = 0;
int num;
pParam = argv[1];
for(int i = 1; i <= nParams; i++ ){
pParam = argv[i]; // Copy the pointer to the first parameter passed
num = atoi(pParam);
sum += num;
printf("pParam = %s, sum = %d\n", pParam, sum);
printf("The sum of numbers given is: %d\n", sum);

}

```

– End of Program –

/* Type in Command prompt (Use any one compiler) */

```

cc Add_num.c      // cc compiler
./a.out

```

```

gcc -o Add_num Add_num.c // gcc compiler
./Add_num 1 2 3 4

```

ii. Minimum.c /* Create this file */

```

#include <stdio.h>
#include <stdlib.h>
#include <limits.h> // required in Linux

int f_Min(int nParams, char * argv[]);
int main(int argc, char *argv[])
{
    printf("Find the minimum of the numbers entered by the user!!!\n");
    if(argc < 2)
    {
        printf("Error: Insufficient parameters!!!\n");
        printf("Give at least one or more valid integer numbers as parameters.\n");
        return -1;
    }
    int nParams = argc -1;
    int min = 0;
    for(int i = 0; i < argc; i++)
    {
        printf(" argv[%d] = %s\n", i, argv[i]);
    }
    min = f_Min(nParams, argv);
    printf("The minimum of the numbers given is: %d\n", min);
    return 0;
}

int f_Min(int nParams, char * argv[]){
    char *pParam;
    int min = INT_MAX;
    int num;
    pParam = argv[1];
    for(int i = 1; i <= nParams; i++ ){
        pParam = argv[i]; // Copy the pointer to the first parameter passed
        num = atoi(pParam);
        if (min > num)
            min = num;
        printf("The min now is %d\n", min);
    }
    return min;
}

```

– End of Program –

/* Type in Command prompt (Use any one compiler) */
gcc -o Minimum Minimum.c

./ Minimum 1 2 3 2 1

iii. Avg.c /* Create this file */

```
#include <stdio.h>
#include <stdlib.h>
float f_Avg(int nParams, char * argv[]);
int main(int argc, char *argv[])
{
    printf("Find the average of the numbers entered by the user!!!\n");
    if(argc < 2){
        printf("Error: Insufficient parameters!!!\n");
        printf("Give at least one or more valid integer numbers as parameters.\n");
        return -1;
    }
    int nParams = argc -1;
    float avg = 0;
    for(int i = 0; i < argc; i++){
        printf(" argv[%d] = %s\n", i, argv[i]);
    }
    average = f_Avg(nParams, argv);
    printf("The average of the numbers given is: %f\n", avg);
    return 0;
}

float f_Avg(int nParams, char * argv[]){
    char *pParam;
    int sum = 0;
    int num;
    pParam = argv[1];
    for(int i = 1; i <= nParams; i++){
        pParam = argv[i]; // Copy the pointer to the first parameter passed
        num = atoi(pParam);
        sum += num;
        printf("pParam = %s, sum = %d\n", pParam, sum);
    }
    return sum/nParams;
}
```

– End of Program –

/* Type in Command prompt (Use any one compiler) */
gcc -o Avg Avg.c

./ Avg 1 2 3 2

1.3) Write a program in C to print the Environment Variable.

env_var.c /* Create this file to show environment variable */

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(int argc, char * argv[], char * envp[])
```

```
{
```

```
printf("Lab2: Print the envp[] contents ...\n");
```

```
for(int i = 0; envp[i] != NULL; i++) {
```

```
printf("%s\n", envp[i]);
```

```
}
```

```
return 0;
```

```
}
```

– End of Program –

/* Type in Command prompt */

```
gcc -o datatype64bit datatype64bit.c
```

```
./ datatype64bit
```

Lab 2 Program: Process Creation and Termination

2. Execute C programs to implement Process system calls using the fork system call, exec, wait, create process, and terminate the process.

This C program demonstrates the use of `fork()`, which is a **system call that creates a child process**.

It also uses `wait()`, **ensuring the parent process waits for the child to complete execution before proceeding**.

Solution:

```
// Create process_syscalls.c
#include<stdio.h> // printf()
#include<stdlib.h> // exit()
#include<sys/types.h> // pid_t
#include<sys/wait.h> // wait()
#include<unistd.h> // fork

int main(int argc, char **argv)
{
    pid_t pid;
    pid = fork();
    if(pid==0)
    {
        printf("This is the child process and pid is %d\n",getpid());
        int i=0;
```

```
for(i=0;i<8;i++)
{
printf("%d\n",i);
}
exit(0);
}
else if(pid > 0)
{
printf("This is the parent process and pid is %d\n",getpid());
int status;
wait(&status);
printf("Child is reaped\n");
}
else
{
printf("Error in forking..\n");
exit(EXIT_FAILURE);
}
return 0;
}
```

Output:

```
gcc -o process_syscalls process_syscalls.c
```

```
./ process_syscalls
```

```
This is the parent process and pid is 4488
```

```
This is the child process and pid is 4489
```

0

1

2

3

4

5

6

7

Child is reaped

Program Name:

`process_syscalls.c`

This program demonstrates:

- Creating a new process using `fork()`
- Executing parent and child processes
- Terminating a process using `exit()`
- Waiting for child process using `wait()`

Step 1: Header Files Used

```
#include<stdio.h>    // printf()
#include<stdlib.h>    // exit()
#include<sys/types.h> // pid_t
#include<sys/wait.h>  // wait()
#include<unistd.h>    // fork(), getpid()
```

✓ Explanation:

Header File	Purpose
<code>stdio.h</code>	Used for <code>printf()</code>
<code>stdlib.h</code>	Used for <code>exit()</code>
<code>sys/types.h</code>	Defines <code>pid_t</code> (process ID type)
<code>sys/wait.h</code>	Used for <code>wait()</code>
<code>unistd.h</code>	Contains <code>fork()</code> and <code>getpid()</code>

Step 2: Main Function

```
int main(int argc, char **argv)
```

- `argc` → Argument count
- `argv` → Argument vector
(Not used in this program, but standard format.)

Step 3: Create Process Using `fork()`

```
pid_t pid;  
pid = fork();
```

✓ What `fork()` does:

- Creates a **new child process**
- Returns:
 - `0` → Inside child process
 - `> 0` → Inside parent process (returns child PID)
 - `< 0` → Error in creating process

After `fork()`, two processes run simultaneously:

1. Parent Process
2. Child Process

Step 4: Child Process Execution

```
if(pid == 0)
{
    printf("This is the child process and pid is %d\n", getpid());
    int i = 0;
    for(i = 0; i < 8; i++)
    {
        printf("%d\n", i);
    }
    exit(0);
}
```

✓ Explanation:

- `pid == 0` → This block executes in **child process**
- `getpid()` → Gets current process ID
- Loop prints numbers from **0 to 7**
- `exit(0)` → Terminates child process successfully

Output from child:

This is the child process and pid is 1234

0
1
2
3
4
5

6
7

Step 5: Parent Process Execution

```
else if(pid > 0)
{
    printf("This is the parent process and pid is %d\n", getpid());
    int status;
    wait(&status);
    printf("Child is reaped\n");
}
```

✓ Explanation:

- `pid > 0` → This block runs in **parent process**
- `getpid()` → Prints parent PID
- `wait(&status):`
 - Parent waits until child finishes
 - Prevents zombie process
- After child terminates → prints "**Child is reaped**"

Output from parent:

This is the parent process and pid is 1233
Child is reaped

Step 6: Error Condition

```
else
{
    printf("Error in forking..\n");
    exit(EXIT_FAILURE);
}
```


- If `fork()` fails → prints error
- `EXIT_FAILURE` → Abnormal termination

Step 7: Program End

`return 0;`

- Indicates successful completion of main program

Overall Execution Flow

1. Program starts
2. `fork()` creates child process
3. Both parent & child execute
4. Child prints numbers and terminates
5. Parent waits using `wait()`
6. Parent prints confirmation
7. Program ends

Process Diagram

Parent Process

|

`fork()`

|

Child Process	Parent Process
Prints 0–7	Waits for child
exit(0)	Prints "Child reaped"

Concepts Covered in This Lab

- Process creation (`fork()`)
- Process ID (`getpid()`)
- Process termination (`exit()`)
- Parent waiting (`wait()`)
- Zombie process prevention

Important Viva Questions

1. What is `fork()`?
2. What does `fork()` return?
3. What is a zombie process?
4. Why do we use `wait()`?
5. Difference between `exit()` and `return`?

Key Takeaways

1. `fork()` creates a child process.

2. The child runs its own execution path.
3. The parent waits for the child to finish using `wait()`.
4. The child prints numbers, then exits.
5. The parent prints a message after the child exits.

This demonstrates basic process creation and synchronization using `fork()` and `wait()`.

LAB 3 -CPU Scheduling – First Come First Served (FCFS)

3. Execute C programs to simulate CPU scheduling algorithms to find turnaround and waiting time.

3.1 First Come First Served (FCFS) scheduling algorithm:

FCFS is the simplest CPU scheduling algorithm.

Rule: The process that arrives first gets executed first.
It follows **FIFO (First In First Out)** principle.

Solution:

// Create process_syscalls.c

```
#include<stdio.h> // printf()
```

```
#include<stdlib.h> // exit()
```

```
#include<sys/types.h> // pid_t
```

```
#include<sys/wait.h> // wait()
```

```
#include<unistd.h> // fork
```

```
int main(int argc, char **argv)
```

```
{
```

```
    pid_t pid;
```

```
    pid = fork();
```

```
    if(pid==0)
```

```
    {
```

```
        printf("This is the child process and pid is %d\n",getpid());
```

```
        int i=0;
```

```
        for(i=0;i<8;i++)
```

```
        {
```

```
printf("%d\n",i);
}
exit(0);
}
else if(pid > 0)
{
printf("This is the parent process and pid is %d\n",getpid());
int status;
wait(&status);
printf("Child is reaped\n");
}
else
{
printf("Error in forking..\n");
exit(EXIT_FAILURE);
}
return 0;
}
```

Output:

Enter total number of processes (maximum 20): 4

Enter process Burst Time:

P[1]: 5

P[2]: 8

P[3]: 2

P[4]: 7

Process	Burst Time	Waiting Time	Turnaround Time
P[1]	5	0	5
P[2]	8	5	13
P[3]	2	13	15
P[4]	7	15	22

Average Waiting Time: 8

Average Turnaround Time: 13

Step 1: Variable Declaration

```
int n, b_time[20], w_time[20], tat[20], avg_wtime=0, avg_tat=0, i, j;
```

✓ Explanation:

Variable	Meaning
n	Number of processes
b_time[20]	Burst time of each process
w_time[20]	Waiting time of each process
tat[20]	Turnaround time of each process
avg_wtime	Average waiting time
avg_tat	Average turnaround time
i, j	Loop variables

Step 2: Input Number of Processes

```
printf("Enter total number of processes(maximum 20): ");
```

```
scanf("%d",&n);
```

User enters total processes (max 20).

Example:

4

Step 3: Input Burst Time

```
printf("\nEnter Process Burst Time:\n");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    printf("P[%d]: ",i+1);
```

```
    scanf("%d",&b_time[i]);
```

```
}
```

Burst Time = Time required by process to execute.

Example Input:

Processes	Burst Time
-----------	------------

P1	5
----	---

P2	8
----	---

P3	2
----	---

P4	7
----	---

Step 4: Calculate Waiting Time

Formula:

Waiting Time of first process = 0

Waiting Time of next process = Sum of burst times of previous processes

$w_time[0] = 0;$

```
for(i=1;i<n;i++)
{
    w_time[i]=0;
    for(j=0;j<i;j++)
        w_time[i] += b_time[j];
}
```

✓ Calculation:

Processes	Calculation	Waiting Time
P1	0	0
P2	5	5
P3	5+8	13
P4	5+8+2	15

Step 5: Calculate Turnaround Time

Formula:

Turnaround Time = Burst Time + Waiting Time

$tat[i] = b_time[i] + w_time[i];$

✓ Calculation:

Processes	Burst	Waiting	Turnaround
P1	5	0	5
P2	8	5	13
P3	2	13	15
P4	7	15	22

Step 6: Calculate Average Times

```
avg_wtime += w_time[i];
```

```
avg_tat += tat[i];
```

```
avg_wtime /= i;
```

```
avg_tat /= i;
```

✓ Calculation:

Average Waiting Time

= $(0 + 5 + 13 + 15) / 4$

= $33 / 4 = 8$

Average Turnaround Time

= $(5 + 13 + 15 + 22) / 4$

= $55 / 4 = 13$

Step 7: Display Output

```
printf("\nProcess\t\tBurst Time\tWaiting Time\tTurnaround Time");
```

Prints process table with all values.

Gantt Chart for Given Example

| P1 | P2 | P3 | P4 |

0 5 13 15 22

Explanation:

- P1 runs from 0–5
- P2 runs from 5–13
- P3 runs from 13–15
- P4 runs from 15–22

Key Concepts in This Lab

- ✓ FCFS is non-preemptive
- ✓ Simple and easy to implement
- ✓ May cause **Convoy Effect** (long process delays short ones)
- ✓ No starvation

Viva Questions

1. What is FCFS scheduling?
2. Is FCFS preemptive or non-preemptive?
3. What is waiting time?
4. What is turnaround time?